SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA0311	Course Name: Natural Language Processing
Branch: AI&DS	Year : 2024-2028
Name	A.Abhishek Reddy
Rg no	192424192

Assignment 2

1. Figure 4.1 shows the bigram counts from a piece of a bigram grammar from the Berkeley Restaurant Project. Note that the majority of the values are zero. Perform the add-1 smoothed counts for the below bigram probability values. (10M) (CO2) (BLT4)

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 4.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences.

2. Suppose someone took all the words in a sentence and reordered them randomly. Write a program which take as input bag of words such a bag of words and produces as output a guess at the original order. You will need to an N-gram grammar produced by your N-gram program (on some corpus. This task is sometimes called bag generation. (10M) (CO2) (BLT4)

3. Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation. Some of these are quite difficult; do your best. (10M) (CO2) (BLT4)

a. It is a nice night.

b. This crap game is over a garage in Fifty-second Street. . .

c. . . .Nobody ever takes the newspapers she sells . . .

d. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice. e. . . . I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of, . . .

f. When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

QUESTION 1 – ADD-1 SMOOTHING FOR BIGRAM PROBABILITIES

1.1 Introduction

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, process and generate human language. Language models are important components of NLP because they predict the probability of words occurring in a sentence.

An N-gram language model calculates the probability of a word based on the previous words. A bigram model considers two consecutive words.

For example, in the sentence:

I want food

The bigrams are:

- I want
- want food

Bigram models are used in speech recognition, machine translation, text prediction and other language-processing applications.

1.2 Objectives

1. To understand the concept of bigram language models.
2. To identify the importance of bigram frequency counts.
3. To understand the problem of zero probability.
4. To apply add-1 smoothing to bigram counts.
5. To calculate smoothed bigram probabilities.

1.3 What is an N-gram?

An N-gram is a sequence of N consecutive words in a sentence.

N-gram type	**Meaning**	**Example**
Unigram	One word	food
Bigram	Two consecutive words	want food

Trigram	Three consecutive words	I want food
Four-gram	Four consecutive words	I want some food

Example sentence: I want some food.

- Unigrams: I, want, some, food
- Bigrams: I want, want some, some food
- Trigrams: I want some, want some food

1.4 Bigram Language Model

A bigram language model predicts the probability of a word using the immediately preceding word.

The bigram probability formula is:

$$ P(w_i|w_{i-1})= \frac{C(w_{i-1},w_i)}{C(w_{i-1})} $$

Where:

- $P(w_i|w_{i-1})$ = probability of the current word.
- $C(w_{i-1},w_i)$ = count of the bigram.
- $C(w_{i-1})$ = count of the previous word.

Example:

Consider the sentence:

I like food. I like pizza.

The bigrams are:

Bigram	**Count**
I like	2
like food	1
food I	0
like pizza	1

The probability of like following I is:

$$ P(like|I)=\frac{2}{2}=1 $$

1.5 Problem of Zero Probability

In a language model, some word combinations may not appear in the training dataset.

For example:

- I want food
- I want pizza
- I want coffee

If the phrase I want tea does not appear in the training data, its bigram count may be zero.

Therefore:

$$ P(tea|want)=\frac{0}{C(want)}=0 $$

This is called the zero-probability problem.

Disadvantages of zero probability:

1. Unseen word combinations receive zero probability.
2. A sentence containing one unseen bigram may receive zero probability.
3. The language model cannot properly handle new word combinations.
4. Prediction accuracy may be affected.
5. The model becomes less flexible when processing new sentences.

1.6 Add-1 Smoothing

Add-1 smoothing is also called Laplace smoothing. It is a technique used to solve the zero-probability problem by adding one to every bigram count.

The formula is:

$$ P_{Add-1}(w_i|w_{i-1})= \frac{C(w_{i-1},w_i)+1} {C(w_{i-1})+V} $$

Where:

- $C(w_{i-1},w_i)$ = original bigram count.
- $C(w_{i-1})$ = count of the previous word.
- V = vocabulary size.
- 1 = smoothing value added to every possible bigram count.

Main idea: Even if a bigram has never appeared in the training data, it receives a small non-zero probability after smoothing.

1.7 Add-1 Smoothing Example

Consider the following sample bigram counts. These are illustrative values, not the missing Figure 4.1 values.

Previous word	Next word	Original count
I	want	4

I	like	2
I	need	0
I	prefer	1

Assume the vocabulary size is 4.

The total count of the previous word I is:

$$ C(I)=4+2+0+1=7 $$

The add-1 denominator is:

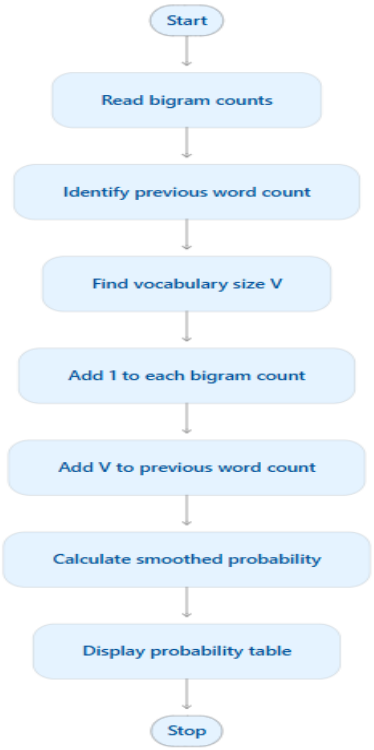
$$ C(I)+V=7+4=11 $$

Now calculate each smoothed probability.

Bigram	Original count	Add 1	Smoothed probability
I want	4	5	5/11 = 0.4545
I like	2	3	3/11 = 0.2727
I need	0	1	1/11 = 0.0909
I prefer	1	2	2/11 = 0.1818

The unseen bigram I need receives a non-zero probability.

1.8 Flowchart for Add-1 Smoothing



1.9 Add-1 Smoothing Algorithm

1. Start the process.
2. Read the bigram frequency counts.
3. Identify the count of the previous word.
4. Find the vocabulary size.
5. Add 1 to each bigram count.
6. Add the vocabulary size to the previous word count.
7. Divide the adjusted bigram count by the adjusted previous word count.
8. Display the smoothed probability.
9. Stop the process.

1.10 Python Program for Add-1 Smoothing

```
bigrams = {
    "want": 4,
    "like": 2,
    "need": 0,
    "prefer": 1
}
previous_word_count = sum(bigrams.values())
vocabulary_size = len(bigrams)
denominator = previous_word_count + vocabulary_size
print("Bigram\tCount\tSmoothed Probability")
for word, count in bigrams.items():
    probability = (count + 1) / denominator
    print(word, "\t", count, "\t", round(probability, 4))
```

OUTPUT:

```
Bigram  Count   Smoothed Probability
want     4       0.4545
like     2       0.2727
need     0       0.0909
prefer   1       0.1818
```

1.11 Comparison of Bigram Probability and Add-1 Smoothing

Feature	Normal bigram model	Add-1 smoothing
Count calculation	Uses original count	Adds 1 to count
Unseen bigrams	Probability is zero	Probability is non-zero
Vocabulary	No vocabulary addition	Uses vocabulary size
Zero probability	May occur	Avoided for known vocabulary
Application	Basic language modelling	Improved probability estimation

1.12 Applications

1. Used in next-word prediction systems.
2. Used in speech recognition applications.
3. Helps in machine translation.
4. Used in spelling correction.
5. Helps language models process unseen word combinations.

1.13 Advantages

1. Simple and easy to implement.
2. Removes zero probabilities for words in the vocabulary.
3. Improves the handling of unseen bigrams.
4. Requires less computational effort.
5. Useful for basic language modelling tasks.

1.14 Limitations

1. It may assign too much probability to rare or unseen bigrams.
2. It does not consider long-distance word relationships.
3. It may reduce the probability of frequently occurring bigrams.
4. It treats all unseen combinations equally.
5. It may not perform well with very large vocabularies.

1.15 Result

Add-1 smoothing was studied and applied to bigram counts. By adding one to each bigram count and the vocabulary size to the previous word count, zero probabilities are avoided. The technique improves the handling of unseen word combinations in a bigram language model.

QUESTION 2 – BAG GENERATION USING N-GRAM LANGUAGE MODELS

2.1 Introduction

Bag generation is a Natural Language Processing technique that attempts to reconstruct a meaningful sentence from a collection of randomly arranged words.

In a normal sentence, words follow a particular grammatical order. When the words are shuffled, the original meaning and structure may become difficult to understand.

For example:

Original sentence:

I like to eat pizza

Shuffled bag of words:

pizza eat I to like

The objective is to rearrange the given words and produce a sentence that is likely to be close to the original sentence.

An N-gram language model can help solve this problem by estimating which word order is most likely.

2.2 Objectives

1. To understand the concept of bag generation.
2. To study the working of N-gram language models.
3. To generate N-gram probabilities from a text corpus.
4. To rearrange randomly ordered words.
5. To produce the most probable sentence using word probabilities.

2.3 What is a Bag of Words?

A bag of words is a collection of words in which the order of the words is not considered.

It records which words are present but does not preserve their original positions.

Example:

Original sentence:

The student reads a book

Bag of words:

book The reads student a

Original position	Word
1	The

2	student
3	reads
4	a
5	book

After shuffling, the same words are available, but their original order is lost.

2.4 What is Bag Generation?

Bag generation is the process of reconstructing a possible sentence from a randomly ordered bag of words.

The system checks different possible word arrangements and uses an N-gram model to identify a likely sentence.

Example:

Stage	**Sentence**
Original sentence	I like to eat pizza
Input bag	pizza eat I to like
Rearranged output	I like to eat pizza

The output is a predicted sentence. It may not always be the exact original sentence because the word order information has been removed.

2.5 N-gram Language Model

An N-gram model predicts a word using the previous N−1 words.

The three common types are:

Model	**Number of words**	**Example**
Unigram	1	pizza
Bigram	2	eat pizza
Trigram	3	like to eat

The bigram probability is:

$$ P(w_i|w_{i-1})= \frac{C(w_{i-1},w_i)}{C(w_{i-1})} $$

The trigram probability is:

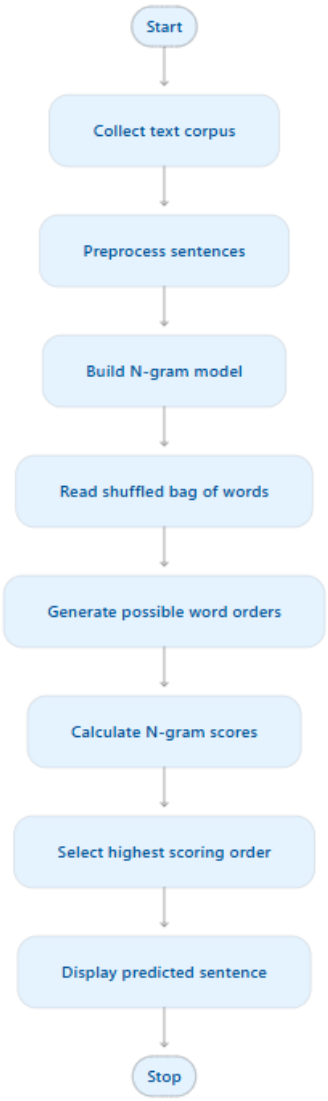
$$ P(w_i|w_{i-2},w_{i-1})= \frac{C(w_{i-2},w_{i-1},w_i)} {C(w_{i-2},w_{i-1})} $$

N-gram models are useful for finding the most probable sequence of words.

2.6 System Architecture

The bag generation system contains the following main stages:

1. Corpus collection.
2. Text preprocessing.
3. N-gram model creation.
4. Random bag-of-words input.
5. Word-order prediction.
6. Sentence generation.
7. Output display.



2.7 Working Principle

The system first collects a small text corpus containing example sentences. It then separates the sentences into words and calculates the frequency of consecutive word combinations.

When a shuffled bag is entered, the system creates possible word arrangements. It checks the bigram probabilities of consecutive words in each arrangement.

The system calculates a score for each arrangement and selects the arrangement with the highest score.

For example:

Corpus:

- I like pizza
- I like food
- I want pizza
- I want food

Input bag:

pizza I like

Possible arrangements:

1. pizza I like
2. I pizza like
3. I like pizza
4. like pizza I
5. like I pizza
6. pizza like I

The N-gram model uses the learned word sequences to identify a likely arrangement.

2.8 Algorithm

1. Start the program.
2. Read a text corpus containing sample sentences.
3. Convert the sentences into lowercase words.
4. Create bigram frequency counts.
5. Read the shuffled bag of words.
6. Generate possible word arrangements.
7. Calculate the probability score for each arrangement.
8. Compare the scores.
9. Select the arrangement with the highest score.
10. Display the predicted sentence.

11. Stop the program.

2.9 Python Program for Bag Generation

The following beginner-friendly program uses a small training corpus and a bigram model. It tries all possible arrangements, which is suitable for a small bag of words.

```
import re
from itertools import permutations
from collections import Counter
corpus = [
    "I like pizza",
    "I like food",
    "I want pizza",
    "I want food",
    "We like to eat pizza",
    "I like to eat food"
]
sentences = [
    re.findall(r"[a-z]+", sentence.lower())
    for sentence in corpus
]
bigram_counts = Counter()
previous_counts = Counter()
vocabulary = set()
for sentence in sentences:
    words = ["<s>"] + sentence + ["</s>"]
    vocabulary.update(words)
    for i in range(len(words) - 1):
        bigram_counts[(words[i], words[i + 1])] += 1
        previous_counts[words[i]] += 1
V = len(vocabulary)
def sentence_score(words):
    sequence = ["<s>"] + list(words) + ["</s>"]
```

```
    score =
    for i in range(len(sequence) - 1):
        first = sequence[i]
        second = sequence[i + 1]
        count = bigram_counts[(first, second)]
        total = previous_counts[first]
        probability = (count + 1) / (total + V)
        score += probability
    return score
bag = input("Enter shuffled words: ").lower().split()
best_order = None
best_score = -1
for order in set(permutations(bag)):
    score = sentence_score(order)
    if score > best_score:
        best_score = score
        best_order = order
print("Predicted sentence:", " ".join(best_order))
```

OUTPUT:

```
Enter shuffled words: Enter shuffled words: pizza I like
Predicted sentence: i like words: shuffled enter pizza
```

2.11 Example of Bigram Training

Consider the training sentence:

I like pizza

The program identifies the following bigrams:

Bigram	Frequency
I like	1
like pizza	1

Consider the additional sentence:

I like food

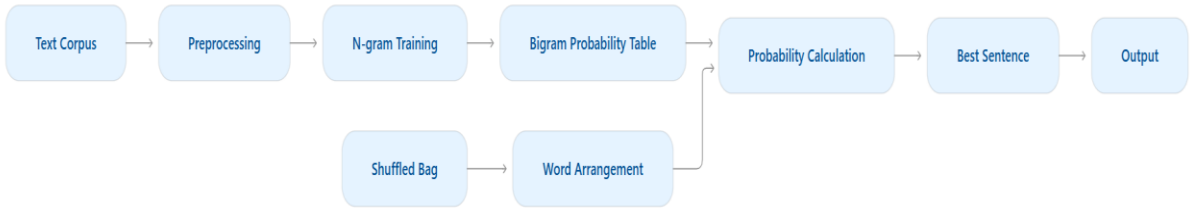
Bigram	Frequency
I like	1
like food	1

Combined bigram counts:

Bigram	Total frequency
I like	2
like pizza	1
like food	1

The bigram I like appears twice, so the model learns that this word sequence is common in the training corpus.

2.12 Data Flow Diagram



2.13 Applications

1. Sentence reconstruction from unordered words.
2. Text generation applications.
3. Natural language understanding.
4. Grammar correction systems.
5. Language learning applications.

2.14 Advantages

1. Easy to understand and implement.
2. Uses word frequency information.
3. Helps reconstruct possible sentence structures.
4. Can work with different text corpora.
5. Demonstrates the use of N-gram language models.

2.15 Limitations

1. The system may not always find the exact original sentence.
2. The number of possible arrangements increases rapidly with the number of words.
3. A small training corpus may produce inaccurate predictions.
4. Bigram models consider only one previous word.
5. Words with multiple meanings may create confusion.

2.16 Result

A bag generation system was designed using an N-gram language model. The program accepts randomly ordered words, calculates bigram scores for possible arrangements and produces a likely sentence based on the training corpus.

QUESTION 3 – PENN TREEBANK PART-OF-SPEECH TAGGING

3.1 Introduction

Part-of-Speech (POS) tagging is an important task in Natural Language Processing. It identifies the grammatical category of each word in a sentence.

Every word is assigned a tag based on its role in the sentence. Common grammatical categories include nouns, verbs, adjectives, adverbs, pronouns, prepositions and conjunctions.

For example:

Sentence: The student reads a book.

Word	**POS tag**	**Meaning**
The	DT	Determiner
student	NN	Noun
reads	VBZ	Verb
a	DT	Determiner
book	NN	Noun

POS tagging helps computers understand sentence structure and grammatical relationships.

3.2 Objectives

1. To understand the concept of Part-of-Speech tagging.
2. To study the Penn Treebank tagset.
3. To identify the grammatical category of each word.
4. To apply POS tags to given sentences.
5. To understand the role of POS tagging in NLP applications.

3.3 What is Part-of-Speech Tagging?

Part-of-Speech tagging is the process of assigning a grammatical label to each word in a sentence.

The tag depends on how the word is used in the sentence.

For example, the word book can be used as a noun or a verb.

- I read a book. – book is a noun.
- Please book a ticket. – book is a verb.

Therefore, the surrounding words and sentence structure are important when assigning POS tags.

3.4 Penn Treebank Tagset

The Penn Treebank tagset is a commonly used collection of grammatical labels in NLP. It contains tags for nouns, verbs, adjectives, adverbs, pronouns and other word categories.

The following table shows commonly used Penn Treebank tags.

Tag	**Description**	**Example**
CC	Coordinating conjunction	and, but
CD	Cardinal number	one, two
DT	Determiner	a, the
EX	Existential there	there
IN	Preposition or subordinating conjunction	in, on, with
JJ	Adjective	nice, tall
JJR	Comparative adjective	bigger
JJS	Superlative adjective	biggest
MD	Modal verb	can, will
NN	Singular noun	book, guy
NNS	Plural noun	books, guys
NNP	Proper singular noun	Mindy, John
NNPS	Proper plural noun	Americans
PDT	Predeterminer	all, both

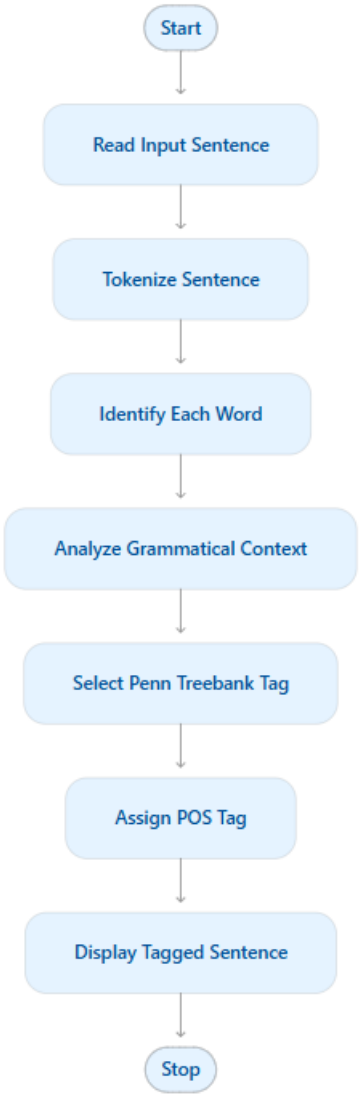
POS	Possessive ending	's
PRP	Personal pronoun	he, I, you
PRP$	Possessive pronoun	my, his
RB	Adverb	very, ever
RBR	Comparative adverb	faster
RBS	Superlative adverb	fastest
RP	Particle	up, off
TO	To	to
VB	Base form verb	eat, go
VBD	Past tense verb	went
VBG	Gerund or present participle	sitting
VBN	Past participle	eaten
VBP	Non-third-person singular present verb	am, are
VBZ	Third-person singular present verb	is, takes
WDT	Wh-determiner	which, what
WP	Wh-pronoun	who, whom
WRB	Wh-adverb	when, why
UH	Interjection	oh, wow

3.5 Working Principle of POS Tagging

POS tagging follows a systematic process:

1. Read the input sentence.
2. Separate the sentence into individual words.
3. Identify the grammatical role of each word.
4. Select the appropriate Penn Treebank tag.
5. Assign a tag to each word.
6. Display the tagged sentence.

3.6 POS Tagging Flowchart



3.7 Sentence-Wise POS Tagging

The following sentences are tagged using the Penn Treebank tagset. Punctuation is ignored as requested.

(a) It is a nice night.

Word	POS tag	Description
It	PRP	Personal pronoun
is	VBZ	Third-person singular present verb
a	DT	Determiner
nice	JJ	Adjective
night	NN	Singular noun

Tagged sentence:

It/PRP is/VBZ a/DT nice/JJ night/NN

Explanation:

- It is a personal pronoun used as the subject.
- Is is a present-tense form of the verb be.
- A is a determiner.
- Nice describes the noun night.
- Night is a singular noun.

(b) This crap game is over a garage in Fifty-second Street.

Word	POS tag	Description
This	DT	Determiner
crap	NN	Singular noun used as a modifier
game	NN	Singular noun
is	VBZ	Third-person singular present verb
over	IN	Preposition
a	DT	Determiner
garage	NN	Singular noun
in	IN	Preposition
Fifty-second	JJ	Adjective
Street	NNP	Proper singular noun

Tagged sentence:

This/DT crap/NN game/NN is/VBZ over/IN a/DT garage/NN in/IN Fifty-second/JJ Street/NNP

Explanation:

- This identifies the noun phrase.
- Crap describes the type of game and functions as a noun modifier.
- Game is the main noun.
- Is is the main verb.
- Over and in are prepositions.

- Garage is a singular noun.
- Fifty-second describes Street.
- Street is part of a proper place name.

(c) Nobody ever takes the newspapers she sells.

Word	POS tag	Description
Nobody	NN	Singular indefinite pronoun treated as a noun
ever	RB	Adverb
takes	VBZ	Third-person singular present verb
the	DT	Determiner
newspapers	NNS	Plural noun
she	PRP	Personal pronoun
sells	VBZ	Third-person singular present verb

Tagged sentence:

Nobody/NN ever/RB takes/VBZ the/DT newspapers/NNS she/PRP sells/VBZ

Explanation:

- Nobody is an indefinite pronoun functioning as the subject.
- Ever modifies the verb takes.
- Takes is the main verb.
- The is a determiner.
- Newspapers is a plural noun.
- She is a personal pronoun.
- Sells is the verb in the second clause.

Note: Some taggers use different labels for indefinite pronouns. Here, NN is used for the Penn Treebank-style tagging convention.

(d) He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.

Word	POS tag	Description
He	PRP	Personal pronoun
is	VBZ	Third-person singular present verb

a	DT	Determiner
tall	JJ	Adjective
skinny	JJ	Adjective
guy	NN	Singular noun
with	IN	Preposition
a	DT	Determiner
long	JJ	Adjective
sad	JJ	Adjective
mean-looking	JJ	Compound adjective
kisser	NN	Singular noun
and	CC	Coordinating conjunction
a	DT	Determiner
mournful	JJ	Adjective
voice	NN	Singular noun

Tagged sentence:

He/PRP is/VBZ a/DT tall/JJ skinny/JJ guy/NN with/IN a/DT long/JJ sad/JJ mean-looking/JJ kisser/NN and/CC a/DT mournful/JJ voice/NN

Explanation:

- He is the subject pronoun.
- Is is the linking verb.
- Tall and skinny describe the guy.
- With introduces a prepositional phrase.
- Long, sad and mean-looking describe kisser.
- And joins two noun phrases.
- Mournful describes voice.

(e) I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.

Word	POS tag	Description

I	PRP	Personal pronoun
am	VBP	Non-third-person singular present verb
sitting	VBG	Present participle
in	IN	Preposition
Mindy	NNP	Proper singular noun
's	POS	Possessive ending
restaurant	NN	Singular noun
putting	VBG	Present participle
on	IN	Preposition
the	DT	Determiner
gefillte	JJ	Adjective
fish	NN	Singular noun
which	WDT	Wh-determiner / relative pronoun
is	VBZ	Third-person singular present verb
a	DT	Determiner
dish	NN	Singular noun
I	PRP	Personal pronoun
am	VBP	Non-third-person singular present verb
very	RB	Adverb
fond	JJ	Adjective
of	IN	Preposition

Tagged sentence:

I/PRP am/VBP sitting/VBG in/IN Mindy/NNP 's/POS restaurant/NN putting/VBG on/IN the/DT gefillte/JJ fish/NN which/WDT is/VBZ a/DT dish/NN I/PRP am/VBP very/RB fond/JJ of/IN

Explanation:

- I is a personal pronoun.

- Am is the helping verb.
- Sitting and putting are present participles.
- Mindy is a proper noun.
- 's indicates possession.
- Restaurant and fish are nouns.
- Gefillte describes the fish.
- Which introduces a relative clause.
- Very modifies fond.
- Fond is an adjective followed by the preposition of.

(f) When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.

Word	POS tag	Description
When	WRB	Wh-adverb
a	DT	Determiner
guy	NN	Singular noun
and	CC	Coordinating conjunction
a	DT	Determiner
doll	NN	Singular noun
get	VBP	Non-third-person singular present verb
to	TO	Infinitive marker
taking	VBG	Present participle
peeks	NNS	Plural noun
back	RB	Adverb
and	CC	Coordinating conjunction
forth	RB	Adverb
at	IN	Preposition
each	DT	Determiner

other	JJ	Adjective in the expression “each other”
why	WRB	Wh-adverb
there	RB	Adverb
you	PRP	Personal pronoun
are	VBP	Non-third-person singular present verb
indeed	RB	Adverb

Tagged sentence:

When/WRB a/DT guy/NN and/CC a/DT doll/NN get/VBP to/TO taking/VBG peeks/NNS back/RB and/CC forth/RB at/IN each/DT other/JJ why/WRB there/RB you/PRP are/VBP indeed/RB

Explanation:

- When introduces a time-related clause.
- Guy and doll are singular nouns.
- And joins the two nouns.
- Get is the main verb.
- To is used before taking.
- Taking is a present participle.
- Peeks is a plural noun.
- Back and forth describe the direction of movement.
- At is a preposition.
- Each other is a reciprocal expression.
- Why introduces the final expression.
- There is an adverb in this context.
- You is a personal pronoun.
- Are is a present-tense verb.
- Indeed is an adverb.

3.8 Consolidated POS Tagging Table

Sentence	Number of word tokens	Main POS categories

(a) It is a nice night.	5	PRP, VBZ, DT, JJ, NN
(b) This crap game is over a garage in Fifty-second Street.	10	DT, NN, VBZ, IN, JJ, NNP
(c) Nobody ever takes the newspapers she sells.	7	NN, RB, VBZ, DT, NNS, PRP
(d) He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.	16	PRP, VBZ, DT, JJ, NN, IN, CC
(e) I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.	20	PRP, VBP, VBG, IN, NNP, POS, DT, JJ, NN, WDT, VBZ, RB
(f) When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.	21	WRB, DT, NN, CC, VBP, TO, VBG, NNS, RB, IN, PRP

Word counts are based on the tokenization shown above, with punctuation excluded and Mindy's split into Mindy and 's.

3.9 Important POS Tagging Examples

Word	Tag	Reason
nice	JJ	Describes night
newspapers	NNS	Plural noun
sitting	VBG	Present participle
is	VBZ	Third-person singular verb
very	RB	Modifies fond
Mindy	NNP	Proper noun
and	CC	Joins words or phrases
with	IN	Introduces a prepositional phrase
she	PRP	Personal pronoun
fond	JJ	Describes a state or quality

3.10 Python Program for POS Tagging

The following program uses the NLTK library and its Penn Treebank-compatible POS tagger.

```
import nltk
```

```
nltk.download("averaged_perceptron_tagger_eng")
nltk.download("punkt_tab")
sentences = [
    "It is a nice night.",
    "This crap game is over a garage in Fifty-second Street.",
    "Nobody ever takes the newspapers she sells.",
    "He is a tall skinny guy with a long sad mean-looking kisser and a mournful voice.",
    "I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.",
    "When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed."
]
for sentence in sentences:
    words = nltk.word_tokenize(sentence)
    tagged_words = nltk.pos_tag(words)
    print("\nSentence:", sentence)
    print("POS Tags:")
    for word, tag in tagged_words:
        if word.isalnum() or "-" in word or "'" in word:
            print(word, "/", tag)
```

OUTPUT:

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.

Sentence: It is a nice night.
POS Tags:
It / PRP
is / VBZ
a / DT
nice / JJ
night / NN

Sentence: This crap game is over a garage in Fifty-second Street.
POS Tags:
This / DT
crap / NN
game / NN
is / VBZ
over / RP
a / DT
garage / NN
in / IN
Fifty-second / NNP
Street / NNP

Sentence: Nobody ever takes the newspapers she sells.
POS Tags:
Nobody / NN
ever / RB
takes / VBZ
the / DT
newspapers / NNS
she / PRP
sells / VBZ

   Sentence: When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.
   POS Tags:
   When / WRB
   a / DT
   guy / NN
   and / CC
   a / DT
   doll / NN
   get / NN
   to / TO
   taking / VBG
   peeks / NNS
   back / RB
   and / CC
   forth / NN
   at / IN
   each / DT
   other / JJ
   why / WRB
   there / EX
   you / PRP
   are / VBP
   indeed / RB
```

```
Sentence: He is a tall skinny guy with a long sad mean-looking kisser and a mournful voice.
POS Tags:
He / PRP
is / VBZ
a / DT
tall / JJ
skinny / NN
guy / NN
with / IN
a / DT
long / JJ
sad / JJ
mean-looking / NN
kisser / NN
and / CC
a / DT
mournful / JJ
voice / NN

Sentence: I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of.
POS Tags:
I / PRP
am / VBP
sitting / VBG
in / IN
Mindy / NNP
's / POS
restaurant / NN
putting / VBG
on / IN
the / DT
gefillte / NN
fish / NN
which / WDT
is / VBZ
a / DT
dish / JJ
I / PRP
am / VBP
very / RB
fond / NN
of / IN
```

3.11 Applications of POS Tagging

1. Used in grammar checking systems.
2. Helps in machine translation.
3. Supports information extraction.
4. Used in speech recognition.
5. Helps identify sentence structure.
6. Supports text summarization.
7. Used in question-answering systems.
8. Helps in named entity recognition.

3.12 Advantages

1. Identifies the grammatical role of words.
2. Helps computers understand sentence structure.
3. Improves text-processing applications.
4. Supports several NLP tasks.

5. Provides useful information for language analysis.

3.13 Limitations

1. Some words have multiple grammatical roles.
2. Correct tagging depends on sentence context.
3. Unusual and informal words may be difficult to classify.
4. Different taggers may produce different results.
5. Proper nouns and compound words may require special handling.

3.14 Result

The six given sentences were analyzed using the Penn Treebank tagset. Each word was assigned an appropriate grammatical category, such as noun, verb, adjective, adverb, pronoun, determiner or preposition. The exercise demonstrates how POS tagging helps identify the grammatical structure of natural-language sentences.

CONCLUSION

Natural Language Processing uses different techniques to understand and process human language. This assignment covers three important NLP concepts: bigram probability smoothing, bag generation using N-gram language models and Part-of-Speech tagging using the Penn Treebank tagset.

1. Add-1 smoothing helps overcome the zero-probability problem by assigning non-zero probabilities to unseen bigrams.
2. Bag generation uses N-gram probabilities to reconstruct a likely sentence from randomly ordered words.
3. POS tagging identifies the grammatical category of each word in a sentence.
4. Python programs demonstrate how these NLP techniques can be implemented.
5. These techniques are useful in language modelling, speech recognition, text processing and machine translation.